

MEMORIA DE PRACTICAS: RASPBERRY

María Soriano Palacios

1. Describa en detalle cómo funciona el archivo `1_gpio_test.py`, prestando especial atención a las funciones `GPIO.setup` y proporcionando una descripción del significado de cada uno de los grupos de código

¿Cómo funciona `gpiotest.py`?

El archivo `1_gpio_test.py` es un script escrito en Python que simula el funcionamiento de los pines GPIO de una Raspberry Pi. Para ello utiliza la librería `EmulatorGUI`, que nos permite trabajar con una interfaz gráfica donde se pueden activar y desactivar pines como si fueran entradas y salidas reales de una Raspberry.

Al inicio del programa se importan las librerías necesarias como `time` y `traceback`. La línea `'from EmulatorGUI import GPIO'` importa el módulo GPIO del emulador. Este módulo permite configurar pines como entradas o salidas, leer su estado y cambiar su valor.

Después se define la función principal del programa `'Main ()'`. Dentro de esta función se encuentra todo el funcionamiento del script. El código está dentro de un bloque `try`, lo que permite capturar (*catch*) posibles errores y evitar que el programa termine de forma brusca debido a ellos.

Dentro del `main`, configuramos el modo de numeración de los pines (BCM), es decir, los pines se identifican usando la numeración real de los GPIO del chip Broadcom de la Raspberry Pi. Por ejemplo, si referenciamos en nuestro código el número 23, se hará referencia al GPIO23, no al número físico del pin 23 en la placa.

A continuación, se configuran los pines de salida y entrada con `'GPIO.setup()'`. Esta función recibe al menos dos parámetros principales (aunque también podrá recibir parámetros opcionales como `initial` o `pull_up_down`): el primero indica el número del GPIO y el segundo si será entrada o salida. En nuestro caso, los GPIO 4, 17, 18 y 21 se configuran como salidas mediante `'GPIO.OUT'` y los GPIO 23, 15, 24 y 26 se configuran como entradas mediante `'GPIO.IN'`. El programa puede modificar los pines de salida mediante `'GPIO.output()'`,

mientras que los pines de entrada solo se leen mediante '**GPIO.input()**'. Además, en los pines GPIO17, GPIO18 y GPIO21 también se indica '**initial=GPIO.LOW**'. Esto significa que al iniciar el programa esos pines comienzan apagados, es decir, con nivel bajo (LOW). En los pines GPIO23, GPIO15 y GPIO24 se configura con *pull_up_down* (**GPIO.PUD_UP / GPIO.PUD_DOWN**). Esto quiere decir que tienen una resistencia interna de pull-up o pull-down, por lo que su estado por defecto es alto HIGH cuando es pull-up y LOW cuando es pull-down, y cuando se activan, su estado cambia al contrario.

Después de configurar todos los pines, el programa entra en un bucle infinito utilizando '*while (True)*'. Dentro del bucle, se comprueba continuamente el estado de los pines de entrada, revisando si se ha activado alguno de los pines entrada. Mediante una serie de bloques '*if*', se comprueban los pines configurados como entrada y sus estados correspondientes, para ver si han sido activados. El primer bloque, comprueba GPIO23, como este pin tiene pull-up, su valor normal es alto. Cuando se pulsa, pasa a bajo, es decir, **False**. Si esto ocurre, el programa activa GPIO4 y GPIO17, poniéndolos a nivel alto (HIGH) y finalmente, espera un segundo mediante '*time.sleep(1)*'. Repite esta misma lógica tres veces más y, finalmente, el programa incluye un bloque de control de errores:

```
except Exception as ex:
    traceback.print_exc()

finally:
    GPIO.cleanup()
```

Si ocurre algún error durante la ejecución, '*traceback.print_exc()*' muestra información detallada del error. El bloque finally se ejecuta siempre, haya error o no, y llama a '*GPIO.cleanup()*'. Esta función limpia la configuración de los GPIO antes de cerrar el programa, evitando que los pines queden configurados incorrectamente. En resumen, el funcionamiento del programa es el siguiente:

GPIO23 activado → enciende GPIO4 y GPIO17

GPIO15 activado → enciende GPIO18 y GPIO21

GPIO24 activado → apaga GPIO18 y GPIO21

GPIO26 activado → apaga GPIO4 y GPIO17

Código

```
1 gpio_test_solved.py
2 '''
3 -----
4 This script configures GPIO pins and continuously checks if any pins / buttons
5 have been pressed
6 - OUTPUT pins can be turned on or off
7 - INPUT pins read whether there is a signal or not
8
9 So:
10
11 . Pressing GPIO23 → turns on GPIO4 and GPIO17
12
13 . Pressing GPIO15 → turns on GPIO18 and GPIO21
14
15 . Pressing GPIO24 → turns off GPIO18 and GPIO21
16
17 . Pressing GPIO26 → turns off GPIO4 and GPIO17
18 -----
19 '''
20
21 # imports GPIO module from Emulator Script
22 from EmulatorGUI import GPIO
23
24 # real Raspberry Pi GPIO library (commented out)
25 #import RPi.GPIO as GPIO
26
27 # delays like time.sleep()
28 import time
29 # prints detailed error information
30 import traceback
31 # used to make HTTP requests (GET POST DELETE...)
32 import requests
33
34 # defines main
35 def Main():
36     try:
37         # sets GPIO numbering mode to BCM
38         # meaning pins are referenced by their Broadcom GPIO numbers
39         GPIO.setmode(GPIO.BCM)
40         # disables GPIO warning messages.
41         GPIO.setwarnings(False)
42
43         # sets GPIO4 as output pin
44         GPIO.setup(4, GPIO.OUT)
45         # sets GPIO17,18,21 as output and its initial value to LOW (0) - GND
46         GPIO.setup(17, GPIO.OUT, initial = GPIO.LOW)
47         GPIO.setup(18, GPIO.OUT, initial = GPIO.LOW)
48         GPIO.setup(21, GPIO.OUT, initial = GPIO.LOW)
49
50         # sets GPIO23 as input with an internal pull-up resistor
51         # meaning HIGH by default, LOW when activated
52         GPIO.setup(23, GPIO.IN, pull_up_down = GPIO.PUD_UP)
53
54         # sets GPIO15 and GPIO24 as input with an internal pull-down resistor
55         # meaning LOW by default, HIGH when activated
56         GPIO.setup(15, GPIO.IN, pull_up_down = GPIO.PUD_DOWN)
57         GPIO.setup(24, GPIO.IN, pull_up_down = GPIO.PUD_DOWN)
58         # sets GPIO26 as input pin
59         GPIO.setup(26, GPIO.IN)
```


2. ¿Por qué el pin GPIO23 está a "¿ALTO" por defecto, pero GPIO24 está a "BAJO" por defecto?

Explicación

El pin GPIO23 está a nivel alto por defecto porque ha sido configurado con una resistencia interna tipo *pull-up* mediante la instrucción:

```
GPIO.setup(23, GPIO.IN, pull_up_down=GPIO.PUD_UP)
```

Una resistencia *pull-up* fuerza el estado del pin a nivel alto (HIGH) cuando no se está aplicando ninguna señal externa. Esto significa que, por defecto, el pin GPIO23 se encuentra en estado alto y únicamente pasa a nivel bajo (LOW) cuando se activa o se pulsa en el emulador.

Por otro lado, el pin GPIO24 está a nivel bajo por defecto porque ha sido configurado con una resistencia interna tipo *pull-down* mediante la instrucción:

```
GPIO.setup(24, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
```

Una resistencia *pull-down* fuerza el estado del pin a nivel bajo (LOW) cuando no hay ninguna señal externa. Esto implica que GPIO24 se encuentra en estado bajo por defecto y pasa a nivel alto (HIGH) cuando se activa.

En resumen, la diferencia entre ambos pines se debe al tipo de resistencia interna, el pull-up hace que el pin esté en alto por defecto, mientras que el pull-down hace que el pin esté en bajo por defecto.

3. ¿Cuál es la diferencia entre la configuración del pin GPIO17 y el pin GPIO15?

Explicación

La principal diferencia entre la configuración del pin GPIO17 y el pin GPIO15 es que GPIO17 está configurado como salida, mientras que GPIO15 está configurado como entrada.

El pin GPIO17 se configura mediante la instrucción:

```
GPIO.setup(17, GPIO.OUT, initial=GPIO.LOW)
```

Esto significa que GPIO17 es un pin de salida (GPIO.OUT), por lo que el programa puede controlar su estado, estableciéndolo en nivel alto (HIGH) o bajo (LOW) mediante la función *GPIO.output*. Además, se indica un valor inicial, '*initial=GPIO.LOW*', por lo que el pin comienza en estado bajo (LOW), es decir, apagado.

Por otro lado, el pin GPIO15 se configura como entrada mediante:

```
GPIO.setup(15, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
```

Esto significa que GPIO15 es un pin de entrada (GPIO.IN), por lo que el programa no puede modificar directamente su valor, sino que únicamente puede leer su estado mediante la función *GPIO.input*. Además, se configura con una resistencia interna pull-down, lo que hace que su valor por defecto sea bajo (LOW) y pase a alto (HIGH) cuando se activa.

En resumen, GPIO17 es un pin de salida controlado por el programa, mientras que GPIO15 es un pin de entrada cuyo estado depende de la señal recibida.

4. Modifique el código para incluir una instrucción `print ()` en cada instrucción `if` para registrar qué pin se ha activado o desactivado en cada momento.

Explicación

Para registrar qué pin se activa o desactiva en cada momento, se han añadido instrucciones '*print ()*' dentro de cada bloque `if`. De esta forma, cada vez que se activa un pin de entrada, el programa muestra por consola un mensaje indicando qué GPIO se ha activado y qué acciones se realizan sobre los pines de salida.

Esto permite hacer un seguimiento del funcionamiento del programa en tiempo real y facilita la comprobación del comportamiento de los pines.

Código

```
while(True):

    # checks if GPIO23 is LOW
    # 23 uses pull-up, low means input has been activated
    if (GPIO.input(23) == False):
        # Debug print
        print("GPIO23 activated → Turning ON GPIO4 and GPIO17")
        # Sets 4 and 17 to HIGH (turn on)
        GPIO.output(4,GPIO.HIGH)
        GPIO.output(17,GPIO.HIGH)
        # 10 sec delay
        time.sleep(1)

    # checks if GPIO15 is HIGH
    # 15 uses pull-down, high means input has been activated
    if (GPIO.input(15) == True):
        # Debug print
        print("GPIO15 activated → Turning ON GPIO18 and GPIO2")
        # Sets 18 and 21 to HIGH (turn on)
        GPIO.output(18,GPIO.HIGH)
        GPIO.output(21,GPIO.HIGH)
        time.sleep(1)

    # checks if GPIO24 is HIGH
    # 24 uses pull-down, high means input has been activated
    if (GPIO.input(24) == True):
        # Debug print
        print("GPIO24 activated → Turning OFF GPIO18 and GPIO2")
        # Sets 18 and 21 to LOW (turn off)
        GPIO.output(18,GPIO.LOW)
        GPIO.output(21,GPIO.LOW)
        time.sleep(1)

    # checks if GPIO26 is HIGH
    # 26 is configured as input (no pull-up-down), high means input has been activated
    if (GPIO.input(26) == True):
        # Debug print
        print("GPIO26 activated → Turning OFF GPIO4 and GPIO17")
        # Sets 4 and 17 to LOW (turn off)
        GPIO.output(4,GPIO.LOW)
        GPIO.output(17,GPIO.LOW)
        time.sleep(1)
```

```
© usuario@DESKTOP-LN6HJM9:~/python$ python3 1_gpio_test_solved.py
GPIO15 activated → Turning ON GPIO18 and GPIO21
GPIO23 activated → Turning ON GPIO4 and GPIO17
GPIO24 activated → Turning OFF GPIO18 and GPIO21
GPIO15 activated → Turning ON GPIO18 and GPIO21
GPIO24 activated → Turning OFF GPIO18 and GPIO21
```


5. Intente hacer clic en GPIO23 durante 5 segundos. ¿Cuántas veces aparece el registro en la consola?

Explicación

El registro aparece 5 veces en la consola.

```
GPIO23 activated → Turning ON GPIO4 and GPIO17
GPIO23 activated → Turning ON GPIO4 and GPIO17
GPIO23 activated → Turning ON GPIO4 and GPIO17
GPIO23 activated → Turning ON GPIO4 and GPIO17
GPIO23 activated → Turning ON GPIO4 and GPIO17
```

Cuando se activa el GPIO23, el programa entra en el bloque if correspondiente y ejecuta las instrucciones asociadas, incluyendo el *print* y la llamada a *time.sleep(1)*. Esta función hace que el programa se detenga durante 1 segundo en cada iteración.

Dado que el bucle se ejecuta continuamente, mientras el pin GPIO23 permanezca activado, el programa repetirá el bloque if una vez cada segundo. Por tanto, si se mantiene activado durante 5 segundos, el mensaje se mostrará aproximadamente 5 veces.

6. Modifique la primera función "time.sleep" (en la primera sí) cambiando el valor de 1 a 10. Intente hacer clic en GPIO23 durante 5 segundos. ¿Cuántas veces aparece el registro en el Shell?

Explicación

El registro aparece 1 vez en el Shell.

```
GPIO23 activated → Turning ON GPIO4 and GPIO17
```

Al modificar la instrucción *time.sleep(1)* a *time.sleep(10)* en el primer bloque if, el programa se detiene durante 10 segundos cada vez que se activa el GPIO23.

Cuando se pulsa el GPIO23, el programa entra en el if, ejecuta el *print* y luego se queda bloqueado durante 10 segundos debido al *time.sleep(10)*. Durante ese tiempo, el bucle no continúa ejecutándose, por lo que no se vuelven a comprobar los pines.

Como el botón solo se mantiene pulsado durante 5 segundos, el programa no tiene oportunidad de volver a entrar en el bloque if antes de que el pin deje de estar activado.

Código

```
while(True):
    # checks if GPIO23 is LOW
    # 23 uses pull-up, low means input has been activated
    if (GPIO.input(23) == False):
        # Debug print
        print("GPIO23 activated → Turning ON GPIO4 and GPIO17")
        # Sets 4 and 17 to HIGH (turn on)
        GPIO.output(4,GPIO.HIGH)
        GPIO.output(17,GPIO.HIGH)
        # 10 sec delay
        time.sleep(10)
```

7. Copie la configuración del pin GPIO21 al pin GPIO2. Cambie todas las acciones en GPIO21 a GPIO2 (Segunda y tercera declaraciones if). Haga clic 10 veces en el pin GPIO15. ¿Qué le sucede a GPIO2 y GPIO18 cada vez que lo activa? ¿Siempre pasa lo mismo? ¿Por qué?

Explicación

Cuando se hace clic en GPIO15, este pin pasa a nivel alto porque está configurado como entrada con resistencia *pull-down*. Al cumplirse la condición:

```
GPIO.input(15) == True,
```

el programa ejecuta el segundo bloque if, activando GPIO18 y GPIO2:

```
GPIO.output(18, GPIO.HIGH)
```

```
GPIO.output(2, GPIO.HIGH)
```

Por lo tanto, cada vez que se activa GPIO15, GPIO18 y GPIO2 deberían encenderse.

Sin embargo, al hacer clic varias veces en GPIO15, puede que el comportamiento sea indefinido y no siempre ocurra exactamente lo mismo con GPIO2. Esto se debe a que GPIO2 es un pin especial en la Raspberry Pi, ya que se utiliza normalmente para la comunicación

I2C y tiene una resistencia *pull-up* interna asociada. Por esta razón, su comportamiento puede ser diferente al de un GPIO normal como GPIO21.

En cambio, GPIO18 se comportará de forma estable, ya que es un pin GPIO de propósito general estándar. Por eso, al activar GPIO15, GPIO18 se enciende correctamente, mientras que GPIO2 puede presentar un comportamiento impredecible.

En resumen:

- Al pulsar GPIO15 → GPIO18 se activa
- Al pulsar GPIO15 → GPIO2 también debería activarse

Pero GPIO2 puede no comportarse siempre igual porque es un pin especial con pull-up interno asociado al bus I2C

Código

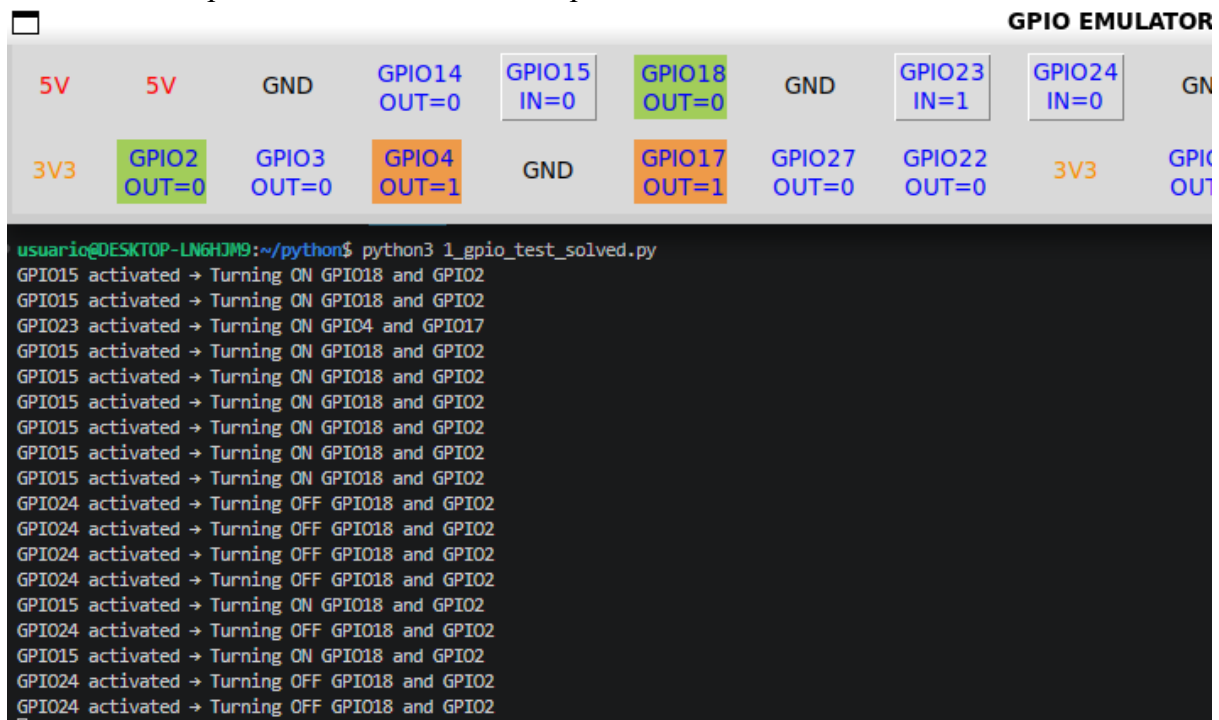
```
45 # sets GPIO17,18,2 as output and its initial value to LOW (0) - GND
46 GPIO.setup(17, GPIO.OUT, initial = GPIO.LOW)
47 GPIO.setup(18, GPIO.OUT, initial = GPIO.LOW)
48 #GPIO.setup(21, GPIO.OUT, initial = GPIO.LOW)
49 GPIO.setup(2, GPIO.OUT, initial=GPIO.LOW)

73 # checks if GPIO15 is HIGH
74 # 15 uses pull-down, high means input has been activated
75 if (GPIO.input(15) == True):
76     # Debug print
77     print("GPIO15 activated → Turning ON GPIO18 and GPIO2")
78     # Sets 8 and 2 to HIGH (turn on)
79     GPIO.output(18,GPIO.HIGH)
80     GPIO.output(2,GPIO.HIGH)
81     time.sleep(1)
82
83 # checks if GPIO24 is HIGH
84 # 24 uses pull-down, high means input has been activated
85 if (GPIO.input(24) == True):
86     # Debug print
87     print("GPIO24 activated → Turning OFF GPIO18 and GPIO2")
88     # Sets 28 and 2 to LOW (turn off)
89     GPIO.output(18,GPIO.LOW)
90     GPIO.output(2,GPIO.LOW)
91     time.sleep(1)
92
```

Resultado

En este caso, el pin GPIO2 se ha comportado de manera estable. Al activar el GPIO15, tanto GPIO18 como GPIO2 se encienden correctamente, y al activar GPIO24, ambos se apagan.

Aunque en una Raspberry Pi real el pin GPIO2 puede presentar particularidades debido a su uso en el bus I2C y a la presencia de resistencias pull-up internas, en el emulador utilizado en esta práctica no se observan comportamientos anómalos.



8. Configure GPIO27 como ENTRADA pull-down. Escriba una nueva declaración if para que, cada vez que GPIO27 esté BAJO, imprima el mensaje "GPIO está bajo", active GPIO2 durante 2 segundos y luego desactívelo.

Explicación

El pin GPIO27 se configura como entrada mediante GPIO.IN y con una resistencia interna *pull-down* (GPIO.PUD_DOWN). Esto significa que su estado por defecto es bajo (LOW).

La condición

```
if GPIO.input(27) == False
```

comprueba si el pin está en nivel bajo. Como el pin está configurado con *pull-down*, esta condición será verdadera mientras el pin no esté activado.

Cuando se cumple la condición, el programa imprime el mensaje "GPIO está bajo" en la consola. A continuación, activa el pin GPIO2 poniéndolo en nivel alto (GPIO.HIGH), lo mantiene activado durante 2 segundos utilizando `time.sleep(2)` y finalmente lo desactiva poniéndolo en nivel bajo (GPIO.LOW).

En resumen, cada vez que GPIO27 se encuentra en estado bajo, el programa muestra un mensaje por consola y activa GPIO2 durante 2 segundos antes de volver a apagarlo.

Código

```
# checks if GPIO27 is LOW
# 27 uses pull-down, low means input has not been activated
if GPIO.input(27) == False:
    # Debug print
    print("GPIO está bajo")
    # Sets 2 to HIGH (turn on)
    GPIO.output(2, GPIO.HIGH)
    time.sleep(2)
    # Sets 2 to LOW (turn off)
    GPIO.output(2, GPIO.LOW)
```

9. ¿Qué sucede cuando ejecuta el script?

Explicación

El pin GPIO27 está configurado como entrada con una resistencia *pull-down* (GPIO.PUD_DOWN). Esto significa que su estado por defecto es bajo (LOW), es decir, **False**.

Como la condición del código es:

```
if GPIO.input(27) == False,
```

esta se cumple constantemente mientras el pin no esté activado. Por este motivo, el programa entra continuamente en el bloque if, imprimiendo el mensaje repetidamente y activando GPIO2 durante 2 segundos en cada iteración.

Sin embargo, cuando se pulsa el GPIO27, su estado cambia a alto (HIGH), es decir, True. En ese momento, la condición deja de cumplirse, por lo que el mensaje deja de imprimirse y el bloque if no se ejecuta.

En resumen:

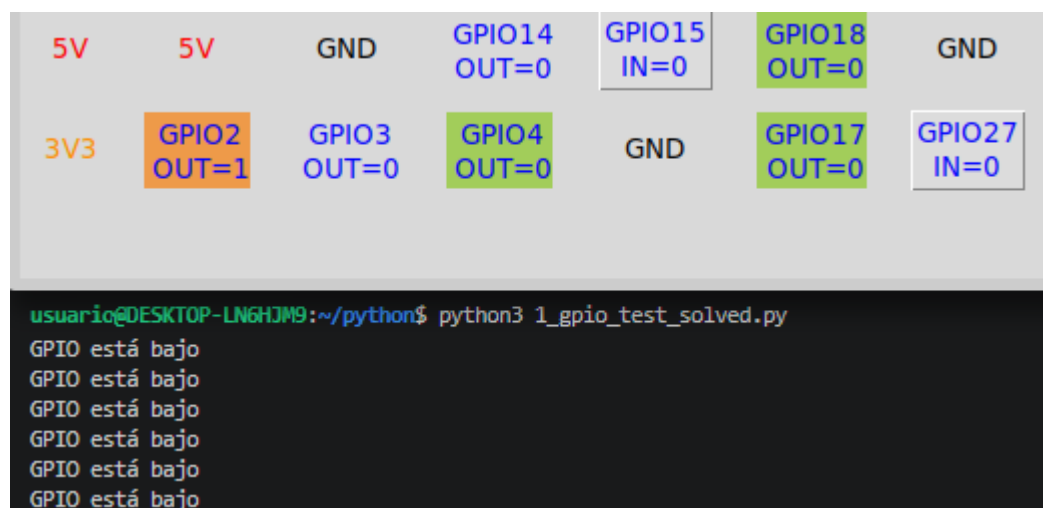
GPIO27 sin pulsar → imprime continuamente "GPIO está bajo"

GPIO27 pulsado → deja de imprimir el mensaje y GPIO2 pasa a LOW

Resultado

Al ejecutar el script, el mensaje "GPIO está bajo" se imprime de forma continua en la consola.

GPIO2 no parece apagarse después de los 2 segundos porque el bloque if está dentro de un bucle infinito. Por ello, el programa activa GPIO2 durante 2 segundos, lo desactiva y, justo después, vuelve a entrar en el mismo if porque GPIO27 sigue estando bajo. **Como consecuencia, GPIO2 se vuelve a encender inmediatamente y parece permanecer encendido de forma constante.**



10. ¿Qué pasaría si cambiamos el pull-down por un pull-up en GPIO27?

Explicación

Si se cambia la configuración del pin GPIO27 de *pull-down* a *pull-up*, su comportamiento se invierte.

Al configurarlo como *pull-up*, el estado por defecto del pin pasa a ser alto (HIGH).

Esto significa que, mientras el pin no esté activado, el pin estará en estado **True**.

Como en el código sigue comprobando la condición anterior:

```
if GPIO.input(27) == False,
```

el bloque if solo se ejecutará cuando el pin GPIO27 pase a estado bajo, es decir, cuando se active.

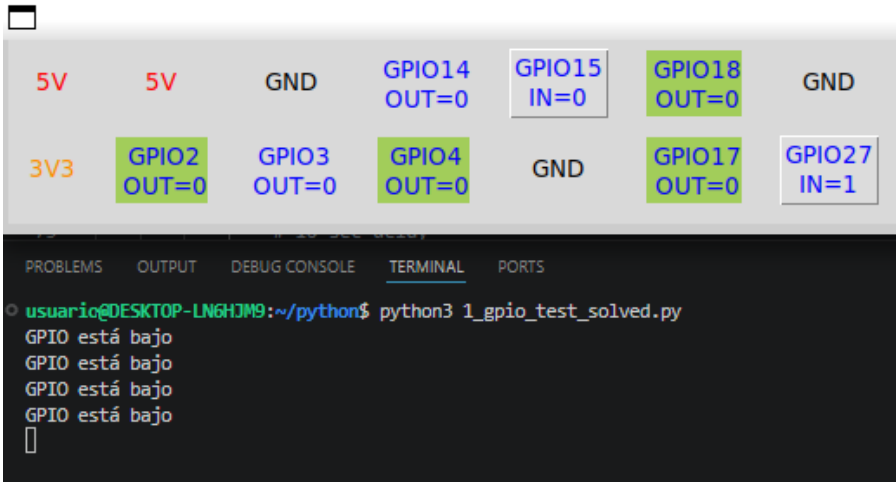
En resumen:

GPIO27 sin pulsar → no entra en la condición if

GPIO27 pulsado → imprime "GPIO está bajo" y activa GPIO2 durante 2 seg

En consecuencia, el mensaje "GPIO está bajo" y la activación de GPIO2 durante 2 segundos solo ocurrirán cuando se pulse GPIO27, y no de forma continua cuando el botón no esté activado.

```
# sets GPIO23, GPIO27 as input with an internal pull-up resistor
# meaning HIGH by default, LOW when activated
GPIO.setup(23, GPIO.IN, pull_up_down = GPIO.PUD_UP)
GPIO.setup(27, GPIO.IN, pull_up_down = GPIO.PUD_UP)
```



```
usuario@DESKTOP-LN6HJM9:~/python$ python3 1_gpio_test_solved.py
GPIO está bajo
GPIO está bajo
GPIO está bajo
GPIO está bajo
[]
```